Exercise 1

NOTE: Use COLAB as editor for performing those programs

Perform following Programs, write & Run the codes and analyze output. Make separate file for each program.

1.
```
print("Hello, World!")
```

2.
```
 if 5 > 2:
  print("Five is greater than two!")
```

3.

Example

Syntax Error:

```
if 5 > 2:
print("Five is greater than two!")
```

4.
```
if 5 > 2:
 print("Five is greater than two!")
if 5 > 2:
        print("Five is greater than two!")
```

6.

Example

Syntax Error:

```
if 5 > 2:
 print("Five is greater than two!")
        print("Five is greater than two!")
```

7.
```
thislist = ["apple", "banana", "cherry"]
print(thislist)
```

8.

Since lists are indexed, lists can have items with the same value:

Example

Lists allow duplicate values:

```
thislist = ["apple", "banana", "cherry", "apple", "cherry"]
print(thislist)
```

9.

Print the second item of the list:

```
thislist = ["apple", "banana", "cherry"]
print(thislist[1])
```

10.

Return the third, fourth, and fifth item:

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]
print(thislist[2:5])
```

11.

Create a Tuple:

```
thistuple = ("apple", "banana", "cherry")
print(thistuple)
```

12.

Create a Set:

```
thisset = {"apple", "banana", "cherry"}
print(thisset)
```

13.

Create a class named MyClass, with a property named x:

```
class MyClass:
  x = 5
```

Create an object named p1, and print the value of x:

```
p1 = MyClass()
print(p1.x)
```

14.

Create a class named Person, use the `__init__()` function to assign values for name and age:

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age

p1 = Person("John", 36)

print(p1.name)
print(p1.age)
```

15.

```python
class Point:
    def __init__(self, x = 0, y = 0):
        self.x = x
        self.y = y

    def add_points(self, other):
        x = self.x + other.x
        y = self.y + other.y
        return Point(x, y)


p1 = Point(1, 2)
p2 = Point(2, 3)

p3 = p1.add_points(p2)

print((p3.x, p3.y))   # Output: (3, 5)
```

16.

```python
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    # overload < operator
    def __lt__(self, other):
        return self.age < other.age

p1 = Person("Alice", 20)
p2 = Person("Bob", 30)

print(p1 < p2)  # prints True
print(p2 < p1)  # prints False
```

17.

```python
x = divmod(5, 2)

print(x)
```